Emanuel Islas Quintos

Comparación de métodos gauss jordan y gauss seidel aplicados a un problema de contexto

Sistemas de ecuaciones utilizados para las pruebas ***3***

Código en Python ***5***

Seudocódigo ***10***

Eliminación de Gauss **10**

Gauss-Jordan **11**

Método de Jacobi **12**

Gauss-Seidel **13**

Salida de algoritmos ***14***

Sistema de ecuaciones 1 **14**

Sistema de ecuaciones 2 **15**

Sistema de ecuaciones 3 **16**

Resultados escritos en libreta ***17***

GEMS OF NUMERICAL METHODS
INFINITY SOLUTION
GAUSS
GAUSS-JORDAN
JACOBI
GAUSS-SEIDEL

Sistemas de ecuaciones utilizados para las pruebas

1. Problema con diagonal dominante

(Ideal para Jacobi o Gauss-Seidel)

En una pequeña empresa se producen **tres tipos de baterías recargables**: A, B y C.

Cada batería requiere diferentes cantidades de tres materiales: **litio, cobre y polímero.**

Los requerimientos por batería son:

Material	Batería A	Batería B	Batería C	Total disponible
Litio	5	1	1	27
Cobre	2	6	1	31
Polímero	1	1	7	34

Si:

- x = baterías tipo A
- y = baterías tipo B
- z = baterías tipo C

Plantea el sistema de ecuaciones que modela el uso total de materiales.

El sistema queda:

$$5x + y + z = 27$$

$$2x + 6y + z = 31$$

$$x + y + 7z = 34$$

En una cafetería se venden **tres combinaciones de desayuno:**
combo A, combo B y combo C.

Durante una mañana se vendieron varios combos y se registró el consumo total de **pan, café y fruta.**

Producto	Combo A	Combo B	Combo C	Total usado
Pan	1	3	2	31
Café	2	1	3	29
Fruta	3	2	1	26

Si:

- x = combos A vendidos
- y = combos B vendidos
- z = combos C vendidos

El sistema es:

$$x + 3y + 2z = 31$$

$$2x + y + 3z = 29$$

$$3x + 2y + z = 26$$

Una empresa de logística controla **10 rutas de transporte** dentro de una ciudad.

Cada ruta consume diferentes cantidades de **10 tipos de combustible sintético** usados para distintos motores.

Durante un día se registró el consumo total de cada combustible.

Sea:

$$x_1, x_2, x_3, \ldots, x_{10}$$

el número de viajes realizados en cada una de las 10 rutas.

El sistema que modela el consumo es:

$$4x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + x_{10} = 40$$

$$x_1 + 5x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + x_{10} = 41$$

$$x_1 + x_2 + 6x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + x_{10} = 42$$

$$x_1 + x_2 + x_3 + 7x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + x_{10} = 43$$

$$x_1 + x_2 + x_3 + x_4 + 8x_5 + x_6 + x_7 + x_8 + x_9 + x_{10} = 44$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + 9x_6 + x_7 + x_8 + x_9 + x_{10} = 45$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + 10x_7 + x_8 + x_9 + x_{10} = 46$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + 11x_8 + x_9 + x_{10} = 47$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + 12x_9 + x_{10} = 48$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + 13x_{10} = 49$$

Código en Python

```
import numpy as np
import time

# METODOS DIRECTOS

def gauss_elimination(A, b):
    A = A.astype(float)
    b = b.astype(float)
```

```python
    n = len(b)

    M = np.hstack([A, b.reshape(-1,1)])

    for i in range(n):
        for j in range(i+1, n):
            factor = M[j][i] / M[i][i]
            M[j] = M[j] - factor * M[i]

    x = np.zeros(n)

    for i in range(n-1, -1, -1):
        x[i] = (M[i][-1] - np.dot(M[i][i+1:n], x[i+1:n])) / M[i][i]

    return x


def gauss_jordan(A, b):
    A = A.astype(float)
    b = b.astype(float)
    n = len(b)

    M = np.hstack([A, b.reshape(-1,1)])

    for i in range(n):
        M[i] = M[i] / M[i][i]

        for j in range(n):
            if j != i:
                factor = M[j][i]
                M[j] = M[j] - factor * M[i]

    return M[:, -1]


# METODOS ITERATIVOS

def jacobi(A, b, tol=1e-8, max_iter=1000):
    n = len(b)
    x = np.zeros(n)
    x_new = np.zeros(n)

    for k in range(max_iter):
        for i in range(n):
            s = np.dot(A[i,:], x) - A[i,i]*x[i]
```

```
            x_new[i] = (b[i] - s) / A[i,i]

        if np.linalg.norm(x_new - x) < tol:
            return x_new, k+1, True

        x = x_new.copy()

    return x, max_iter, False


def gauss_seidel(A, b, tol=1e-8, max_iter=1000):
    n = len(b)
    x = np.zeros(n)

    for k in range(max_iter):
        x_old = x.copy()

        for i in range(n):
            s1 = np.dot(A[i,:i], x[:i])
            s2 = np.dot(A[i,i+1:], x_old[i+1:])
            x[i] = (b[i] - s1 - s2) / A[i,i]

        if np.linalg.norm(x - x_old) < tol:
            return x, k+1, True

    return x, max_iter, False


# FUNCION PARA ANALIZAR METODO
def analizar_metodo(nombre, metodo, A, b, iterativo=False):

    inicio = time.perf_counter()

    if iterativo:
        x, it, conv = metodo(A, b)
    else:
        x = metodo(A, b)
        it = "-"
        conv = True

    fin = time.perf_counter()

    tiempo = fin - inicio
    residual = np.linalg.norm(A @ x - b)
```

```
    print(f"\nMetodo: {nombre}")
    print("Solucion:", np.round(x,6))
    print("Tiempo:", tiempo, "segundos")
    print("Iteraciones:", it)
    print("Convergencia:", conv)
    print("Error residual:", residual)


# DEFINICION DE SISTEMAS

sistemas = []

# SISTEMA 1 (diagonal dominante)

A1 = np.array([
[5,1,1],
[2,6,1],
[1,1,7]
])

b1 = np.array([27,31,34])

sistemas.append(("Sistema 1 (Diagonal dominante)",A1,b1))


# SISTEMA 2 (sin diagonal dominante)

A2 = np.array([
[1,3,2],
[2,1,3],
[3,2,1]
])

b2 = np.array([31,29,26])

sistemas.append(("Sistema 2 (No dominante)",A2,b2))


# SISTEMA 3 (10x10)

A3 = np.array([
[4,1,1,1,1,1,1,1,1,1],
[1,5,1,1,1,1,1,1,1,1],
[1,1,6,1,1,1,1,1,1,1],
[1,1,1,7,1,1,1,1,1,1],
```

```
[1,1,1,1,8,1,1,1,1,1],
[1,1,1,1,1,9,1,1,1,1],
[1,1,1,1,1,1,10,1,1,1],
[1,1,1,1,1,1,1,11,1,1],
[1,1,1,1,1,1,1,1,12,1],
[1,1,1,1,1,1,1,1,1,13]
])

b3 = np.array([40,41,42,43,44,45,46,47,48,49])

sistemas.append(("Sistema 3 (10x10)",A3,b3))


# EJECUCION

for nombre, A, b in sistemas:

    print("\n==================================")
    print(nombre)
    print("==================================")

    analizar_metodo("Gauss", gauss_elimination, A, b)
    analizar_metodo("Gauss-Jordan", gauss_jordan, A, b)
    analizar_metodo("Jacobi", jacobi, A, b, True)
    analizar_metodo("Gauss-Seidel", gauss_seidel, A, b, True)
```

Seudocódigo

Eliminación de **Gauss**

```
Algoritmo EliminacionGauss(A, b)

Entrada:
    A = matriz de coeficientes (n x n)
    b = vector de términos independientes

Formar matriz aumentada [A | b]

Para k = 1 hasta n-1
    Para i = k+1 hasta n
        factor = A[i,k] / A[k,k]
        Para j = k hasta n+1
            A[i,j] = A[i,j] - factor * A[k,j]
        FinPara
    FinPara
FinPara

# Sustitución hacia atrás
Para i = n hasta 1
    suma = 0
    Para j = i+1 hasta n
        suma = suma + A[i,j] * x[j]
    FinPara

    x[i] = (A[i,n+1] - suma) / A[i,i]
FinPara

Salida: x
```

Gauss-Jordan

```
Algoritmo GaussJordan(A, b)

Entrada:
    A = matriz (n x n)
    b = vector independiente

Formar matriz aumentada [A | b]

Para i = 1 hasta n

    pivote = A[i,i]

    # Normalizar fila
    Para j = 1 hasta n+1
        A[i,j] = A[i,j] / pivote
    FinPara

    # Eliminar columna
    Para k = 1 hasta n
        Si k ≠ i entonces

            factor = A[k,i]

            Para j = 1 hasta n+1
                A[k,j] = A[k,j] - factor * A[i,j]
            FinPara

        FinSi
    FinPara

FinPara

La solución es la última columna

Salida: x
    x[i] = (A[i,n+1] - suma) / A[i,i]
FinPara

Salida: x
```

Método de Jacobi

```
Algoritmo Jacobi(A, b, tolerancia, maxIter)

Entrada:
    A = matriz (n x n)
    b = vector independiente

Inicializar:
    x = vector inicial (0,0,...,0)

Para k = 1 hasta maxIter

    Para i = 1 hasta n

        suma = 0

        Para j = 1 hasta n
            Si j ≠ i entonces
                suma = suma + A[i,j] * x[j]
            FinSi
        FinPara

        xNuevo[i] = (b[i] - suma) / A[i,i]

    FinPara

    error = norma(xNuevo - x)

    Si error < tolerancia
        detener
    FinSi

    x = xNuevo

FinPara

Salida: x
```

Gauss-Seidel

```
Algoritmo GaussSeidel(A, b, tolerancia, maxIter)

Entrada:
    A = matriz (n x n)
    b = vector independiente

Inicializar:
    x = vector inicial (0,0,...,0)

Para k = 1 hasta maxIter

    xAnterior = x

    Para i = 1 hasta n

        suma1 = 0
        Para j = 1 hasta i-1
            suma1 = suma1 + A[i,j] * x[j]
        FinPara

        suma2 = 0
        Para j = i+1 hasta n
            suma2 = suma2 + A[i,j] * xAnterior[j]
        FinPara

        x[i] = (b[i] - suma1 - suma2) / A[i,i]

    FinPara

    error = norma(x - xAnterior)

    Si error < tolerancia
        detener
    FinSi

FinPara

Salida: x
```

Salida de algoritmos

Sistema de ecuaciones 1

```
==================================
Sistema 1 (Diagonal dominante)
==================================

Metodo: Gauss
Solucion: [3.994681 3.196809 3.829787]
Tiempo: 0.0008738000178709626 segundos
Iteraciones: -
Convergencia: True
Error residual: 0.0

Metodo: Gauss-Jordan
Solucion: [3.994681 3.196809 3.829787]
Tiempo: 7.399998139590025e-05 segundos
Iteraciones: -
Convergencia: True
Error residual: 3.552713678800501e-15

Metodo: Jacobi
Solucion: [3.994681 3.196809 3.829787]
Tiempo: 0.0003166000824421644 segundos
Iteraciones: 24
Convergencia: True
Error residual: 1.0557642597985494e-08

Metodo: Gauss-Seidel
Solucion: [3.994681 3.196809 3.829787]
Tiempo: 0.0001857000170275569 segundos
Iteraciones: 10
Convergencia: True
Error residual: 3.4416316132156397e-10
```

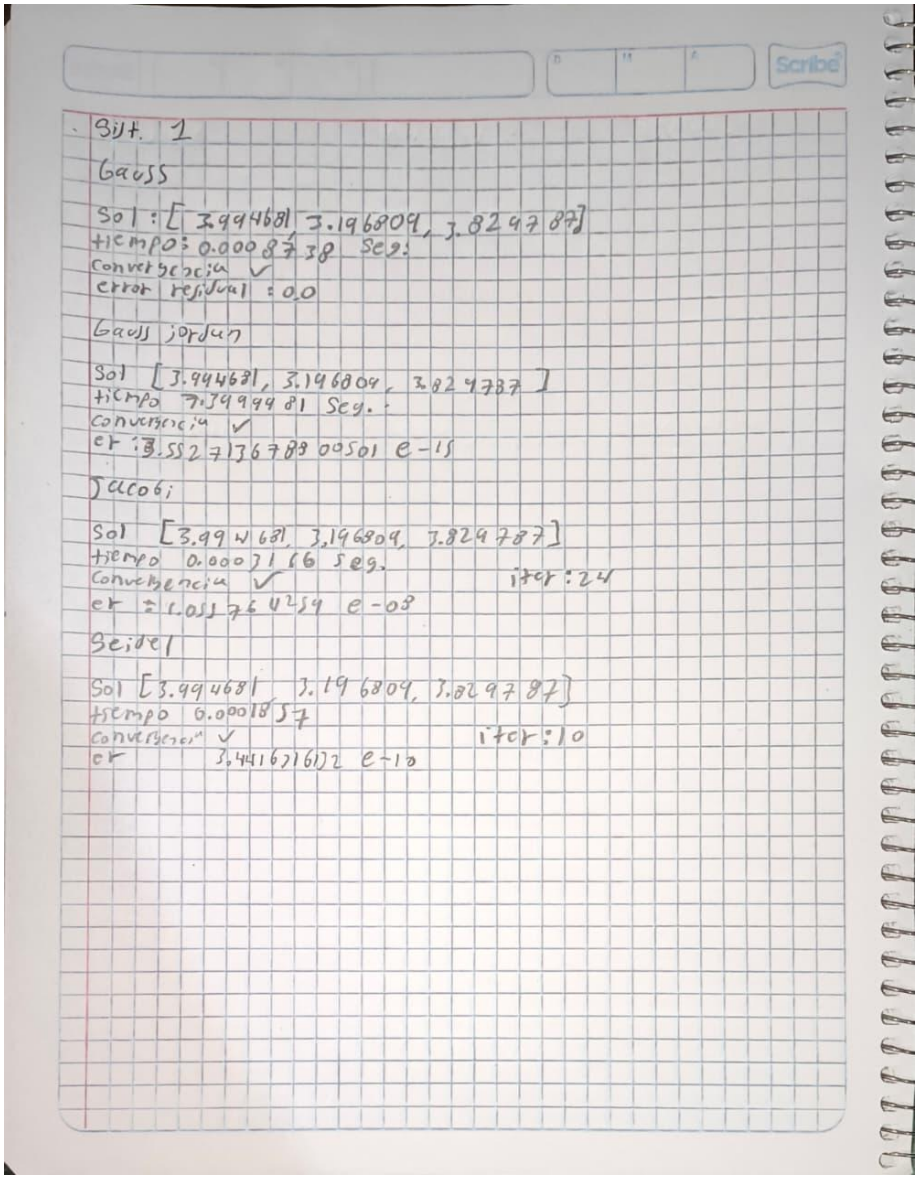
Sistema de ecuaciones 2

```
==================================
Sistema 2 (No dominante)
==================================

Metodo: Gauss
Solucion: [3.111111 5.444444 5.777778]
Tiempo: 6.240000948309898e-05 segundos
Iteraciones: -
Convergencia: True
Error residual: 1.2809491335957507e-14

Metodo: Gauss-Jordan
Solucion: [3.111111 5.444444 5.777778]
Tiempo: 4.12999652326107e-05 segundos
Iteraciones: -
Convergencia: True
Error residual: 7.944109290391274e-15
C:\Escuela\4to\Metodos numericos\problemario gupal\allmetodos.py:57:
RuntimeWarning: invalid value encountered in scalar subtract
  s = np.dot(A[i,:], x) - A[i,i]*x[i]

Metodo: Jacobi
Solucion: [nan nan nan]
Tiempo: 0.01094920001924038 segundos
Iteraciones: 1000
Convergencia: False
Error residual: nan
C:\Escuela\4to\Metodos numericos\problemario gupal\allmetodos.py:78:
RuntimeWarning: invalid value encountered in scalar subtract
  x[i] = (b[i] - s1 - s2) / A[i,i]

Metodo: Gauss-Seidel
Solucion: [nan nan nan]
Tiempo: 0.01480280002579093 segundos
Iteraciones: 1000
Convergencia: False
Error residual: nan
```

Sistema de ecuaciones 3

```
==================================
Sistema 3 (10x10)
==================================

Metodo: Gauss
Solucion: [4.457269 3.592952 3.074361 2.728635 2.481687 2.296476 2.152423 2.037181
 1.942892 1.864317]
Tiempo: 0.00014359992928802967 segundos
Iteraciones: -
Convergencia: True
Error residual: 1.4210854715202004e-14

Metodo: Gauss-Jordan
Solucion: [4.457269 3.592952 3.074361 2.728635 2.481687 2.296476 2.152423 2.037181
 1.942892 1.864317]
Tiempo: 0.0001637000823393464 segundos
Iteraciones: -
Convergencia: True
Error residual: 2.3566036520288945e-14

Metodo: Jacobi
Solucion: [-3.15507974e+77 -2.61462730e+77 -2.23225160e+77 -1.94744749e+77
 -1.72709439e+77 -1.55153829e+77 -1.40837904e+77 -1.28940650e+77
 -1.18896860e+77 -1.10304711e+77]
Tiempo: 0.02323439996689558 segundos
Iteraciones: 1000
Convergencia: False
Error residual: 9.536206047141735e+78

Metodo: Gauss-Seidel
Solucion: [4.457269 3.592952 3.074361 2.728635 2.481687 2.296476 2.152423 2.037181
 1.942892 1.864317]
Tiempo: 0.0007069000275805593 segundos
Iteraciones: 18
Convergencia: True
Error residual: 4.666802539656096e-09
```

Resultados escritos en libreta



Sist. 1

Gauss

Sol: [3.944681 3.196809, 3.824787]
tiempo: 0.00087 38 seg:
convergencia ✓
error residual: 0.0

Gauss jordan

Sol [3.944681, 3.196809, 3.824787]
tiempo 7.39999 81 seg.
convergencia ✓
er: 3.552713678300501 e-15

Jacobi

Sol [3.944681, 3.196809, 3.824787]
tiempo 0.00071 66 seg.
convergencia ✓ iter: 24
er = 1.0557642 59 e-08

Seidel

Sol [3.944681 3.196809, 3.824787]
tiempo 0.00018 57
convergencia ✓ iter: 10
er 3.441671612 e-10

Solo escribí uno porque hueva, no necesito aprenderme esos resultados de un problema inventado. Pero, en resumen, cuando un método iterativo (seidel o jacobi) no convergen, siempre se van a hacer todas las iteraciones posibles hasta llegar al límite, el tiempo de ejecución sube bastante y puede arrojar resultados muy distintos o no dar resultados (con el algoritmo programado), por eso es necesario hacer todas las validaciones posibles antes de iniciar las iteraciones, para asegurarse de que el sistema de ecuaciones se pueda resolver por dicho método.